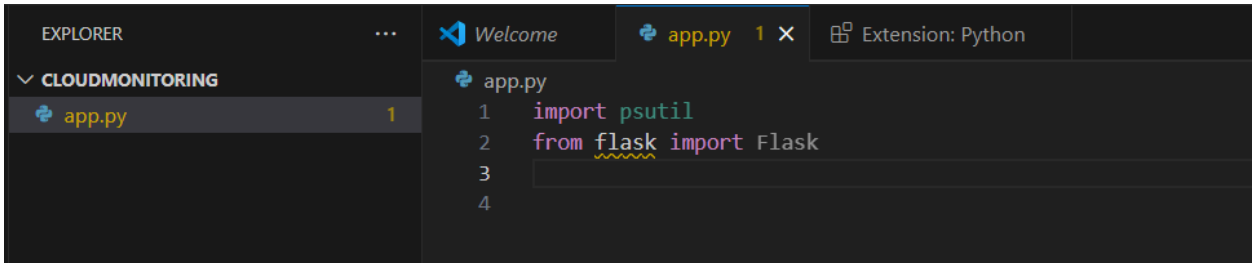
Deploy Cloud Native Monitoring Application on Kubernetes

Creating the App

Psutil: to monitor memory CPU



Flask to create the application itself

```python
import psutil
from flask import Flask

app = Flask(__name__)

@app.route("/")

def index():
    cpu_percent=psutil.cpu_percent()
    mem_percent=psutil.virtual_memory().percent
    message = None
    if cpu_percent > 80 or mem_percent > 80:
        message = "High CPU or Memory Utilization Detected. Please Scale Up."
    return f"CPU Utilization: {cpu_percent} and Memory Utilization: {mem_percent}"
```

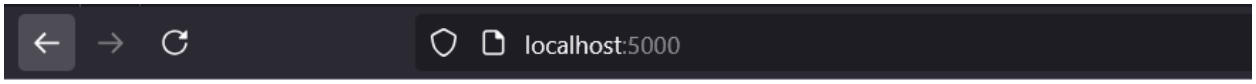
```python
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
```

Welcome | app.py | requirements.txt

requirements.txt

```
Flask==2.2.3
MarkupSafe==2.1.2
Werkzeug==2.2.3
itsdangerous==2.1.2
psutil==5.8.0
plotly==5.5.0
tenacity==8.0.1
boto3==1.9.148
kubernetes==10.0.1
```

```
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> py app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.164:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 121-662-839
```



CPU Utilization: 0.0 and Memory Utilization: 85.7

EXPLORER

CLOUDMONITORING
- templates
  - index.html 9+
- app.py
- requirements.txt

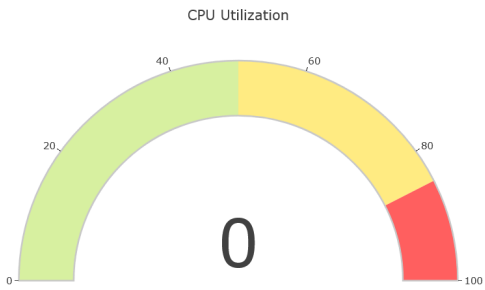
Welcome | app.py | index.html 9+ | requirements.txt | Extension: Python

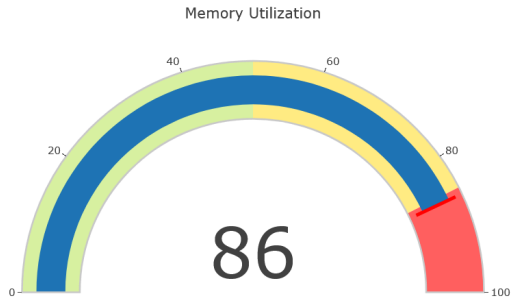
templates > index.html > html

```
<!DOCTYPE html>
<html>
<head>
    <title>System Monitoring</title>
    <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
    <style>
        .plotly-graph-div {
            margin: auto;
            width: 50%;
            background-color: rgba(151, 128, 128, 0.688);
            padding: 20px;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>System Monitoring</h1>
        <div id="cpu-gauge"></div>
        <div id="mem-gauge"></div>
        {% if message %}
```

```python
import psutil
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")

def index():
    cpu_percent=psutil.cpu_percent()
    mem_percent=psutil.virtual_memory().percent
    Message = None
    if cpu_percent > 80 or mem_percent > 80:
        Message = "High CPU or Memory Utilization Detected. Please Scale Up."
    return render_template("index.html", cpu_percent=cpu_percent,mem_percent=mem_percent, message=Message)

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
```

System Monitoring





High CPU or Memory Utilization Detected. Please Scale Up.

Dockerize the Application

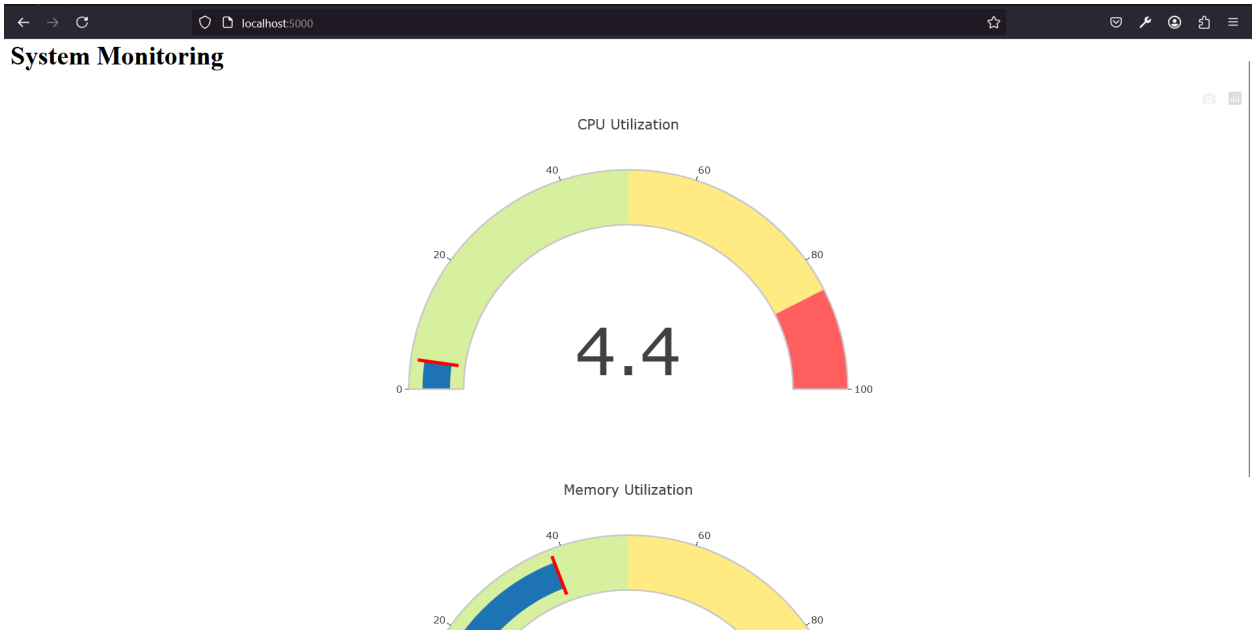
Dockerfile > ...

```dockerfile
FROM python:3.9-slim-buster

WORKDIR /app

COPY requirements.txt .

RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

ENV FLASK_RUN_HOST=0.0.0.0

EXPOSE 5000

CMD [ "flask", "run"]
```

```
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> docker build -t my-flask-app .
[+] Building 3.3s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 247B
 => [internal] load .dockerignore
 => => transferring context: 2B
```

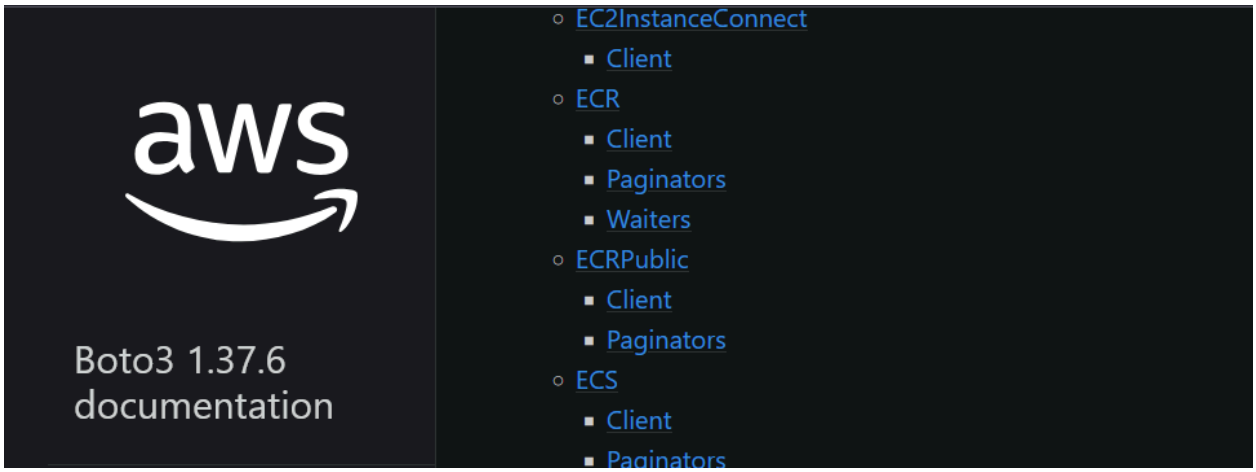
```
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> docker images
REPOSITORY      TAG       IMAGE ID       CREATED          SIZE
my-flask-app    latest    a6462e400c32   10 seconds ago   328MB
```

```
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> docker run -p 5000:5000 a6462e400c32
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [05/Mar/2025 03:10:46] "GET / HTTP/1.1" 200 -
```



Deploying in Kubernetes

ECR
Using Boto3 and Python to create resources in AWS
AWS SDK for python to create configure and manage AWS Resources.



ecr.py > ...

```python
import boto3

ecr_client = boto3.client('ecr')

repository_name = "my-cloud-native-repo"

response = ecr_client.create_repository(
    repositoryName=repository_name)
repository_uri = response['repository']['repositoryUri']
print(repository_uri)
```

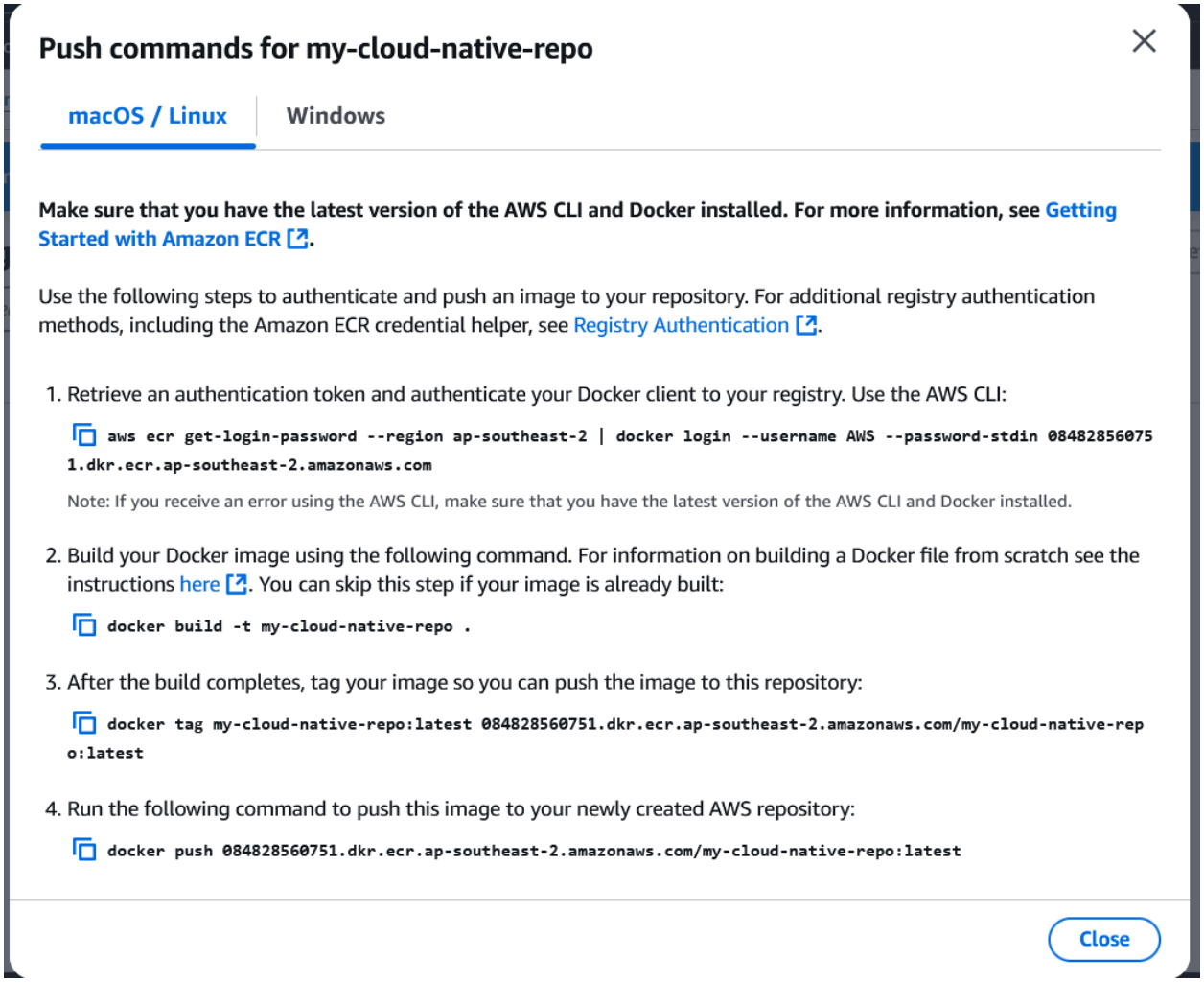
```
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> py ecr.py
084828560751.dkr.ecr.ap-southeast-2.amazonaws.com/my-cloud-native-repo
```

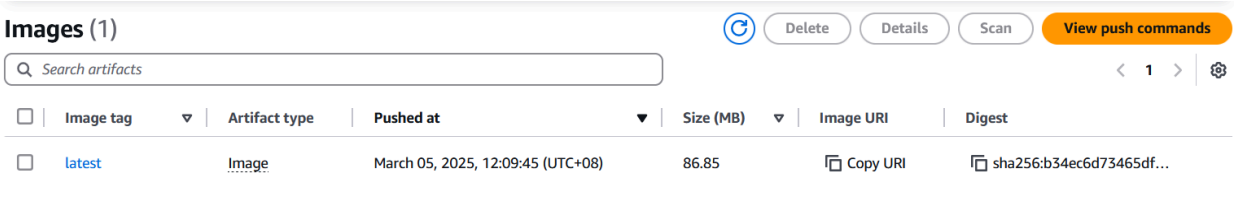
Private repositories (1)

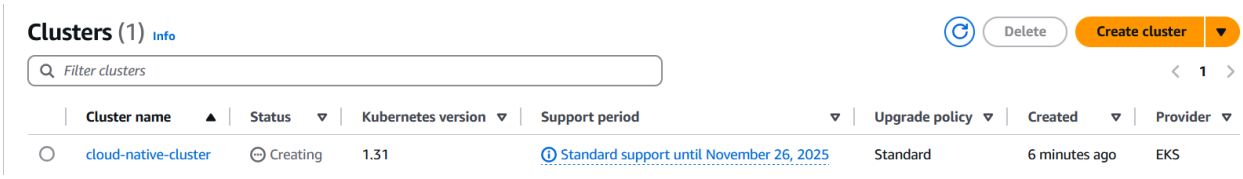
View push commands | Delete | Actions | Create repository

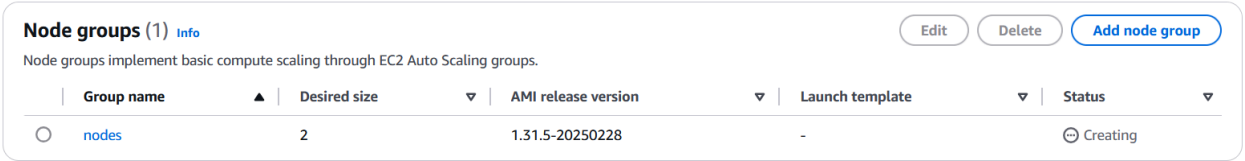
Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
my-cloud-native-repo	084828560751.dkr.ecr.ap-southeast-2.amazonaws.com/my-cloud-native-repo	March 05, 2025, 12:01:11 (UTC+08)	Mutable	AES-256


Push commands for my-cloud-native-repo
macOS / Linux
Windows
Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see Getting Started with Amazon ECR.
Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see Registry Authentication.
1. Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:
aws ecr get-login-password --region ap-southeast-2 | docker login --username AWS --password-stdin 084828560751.dkr.ecr.ap-southeast-2.amazonaws.com
Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.
2. Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions here. You can skip this step if your image is already built:
docker build -t my-cloud-native-repo .
3. After the build completes, tag your image so you can push the image to this repository:
docker tag my-cloud-native-repo:latest 084828560751.dkr.ecr.ap-southeast-2.amazonaws.com/my-cloud-native-repo:latest
4. Run the following command to push this image to your newly created AWS repository:
docker push 084828560751.dkr.ecr.ap-southeast-2.amazonaws.com/my-cloud-native-repo:latest
Close


Images (1)
Delete
Details
Scan
View push commands
Search artifacts
1
Image tag
Artifact type
Pushed at
Size (MB)
Image URI
Digest
latest
Image
March 05, 2025, 12:09:45 (UTC+08)
86.85
Copy URI
sha256:b34ec6d73465df...

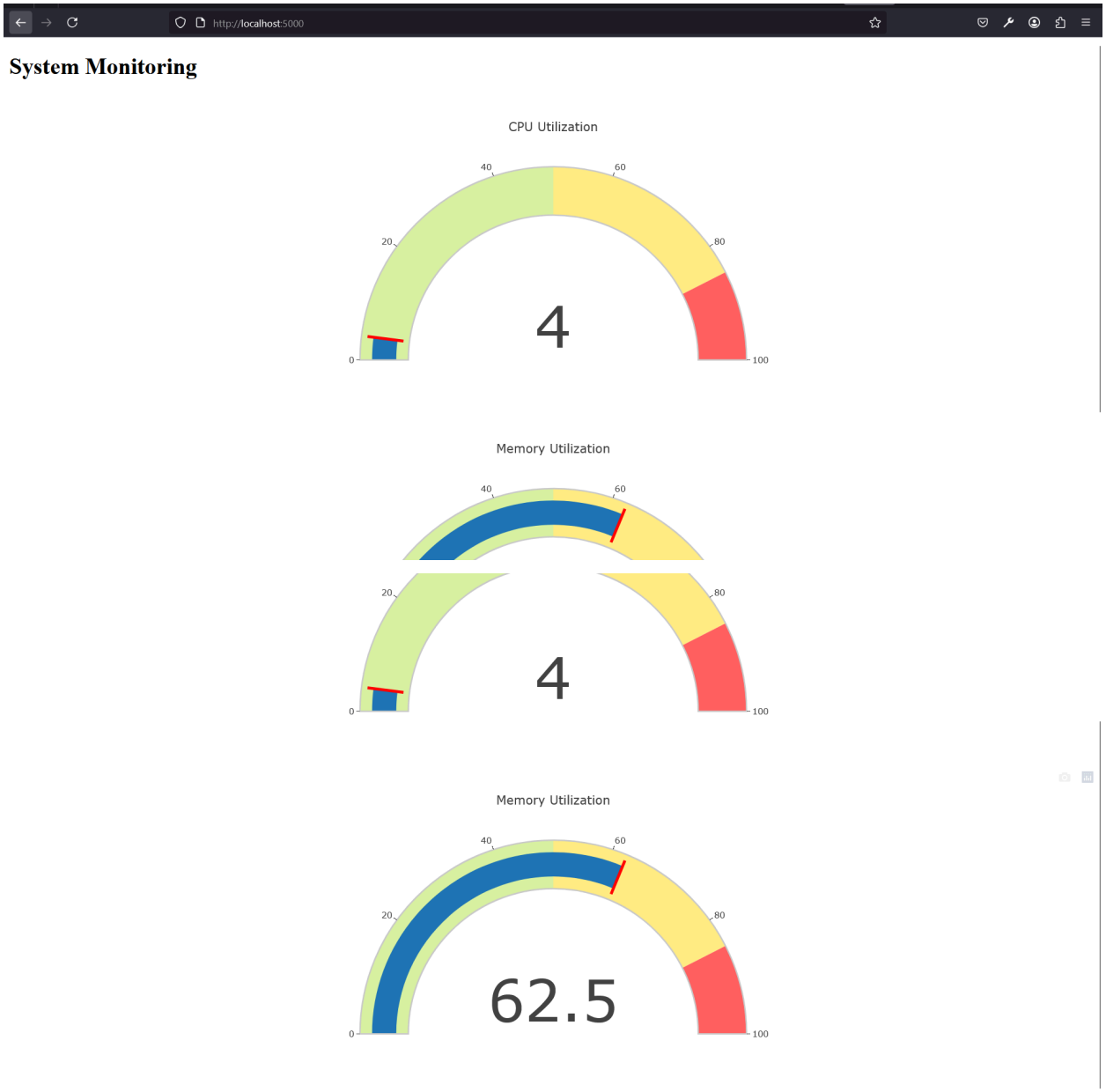

Clusters (1) Info
Delete
Create cluster
Filter clusters
1
Cluster name
Status
Kubernetes version
Support period
Upgrade policy
Created
Provider
cloud-native-cluster
Creating
1.31
Standard support until November 26, 2025
Standard
6 minutes ago
EKS


Node groups (1) Info
Edit
Delete
Add node group
Node groups implement basic compute scaling through EC2 Auto Scaling groups.
Group name
Desired size
AMI release version
Launch template
Status
nodes
2
1.31.5-20250228
-
Creating

Creating a Deployment on Kubernetes
Create a Service on Kubernetes

```python
# Create the deployment
api_instance = client.AppsV1Api(api_client)
api_instance.create_namespaced_deployment(
    namespace="default",
    body=deployment
)

# Define the service
service = client.V1Service(
    metadata=client.V1ObjectMeta(name="my-flask-servic
    spec=client.V1ServiceSpec(
        selector={"app": "my-flask-app"},
        ports=[client.V1ServicePort(port=5000)]
    )
)

# Create the service
api_instance = client.CoreV1Api(api_client)
api_instance.create_namespaced_service(
    namespace="default",
```

```
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> kubectl get pods -n default -w
NAME                          READY   STATUS    RESTARTS   AGE
my-flask-app-8656d96ffd-6nw5r   1/1     Running   0          24m
PS C:\Users\Miguel Huerto\Desktop\cloudmonitoring> kubectl port-forward svc/my-flask-service 5000:5000
Forwarding from 127.0.0.1:5000 -> 5000
Forwarding from [::1]:5000 -> 5000
```



Issues:

Node capacity too small initially set to 2. Moved to 3 then it worked.